

Dungeon report

李昀祐

112550003

Content

- I. Compilation Guide
- II. UML Design
- III. Basic setting and introduction
- IV. Implementation Details
 - i. Movement
 - ii. Showing Status
 - iii. NPC
 - iv. Pick up Items
 - v. Fighting System
 - vi. Game Logic
 - vii. Hunger System
 - viii. Room System
 - ix. Record
- V. Optional enhancement
- VI. Results
- VII. Discussion
- VIII. Conclusion

I. Compilation Guide

To accommodate different operating systems, various compilation methods may be utilized. I provide two different compilation methods for different systems (both have to ensure that all files are located in the same directory.)

First, compile the code using the g++ compiler with the -std=c++11 flag to ensure C++11 compatibility. Use the following command in the terminal:

```
g++ -std=c++11 *.cpp
```

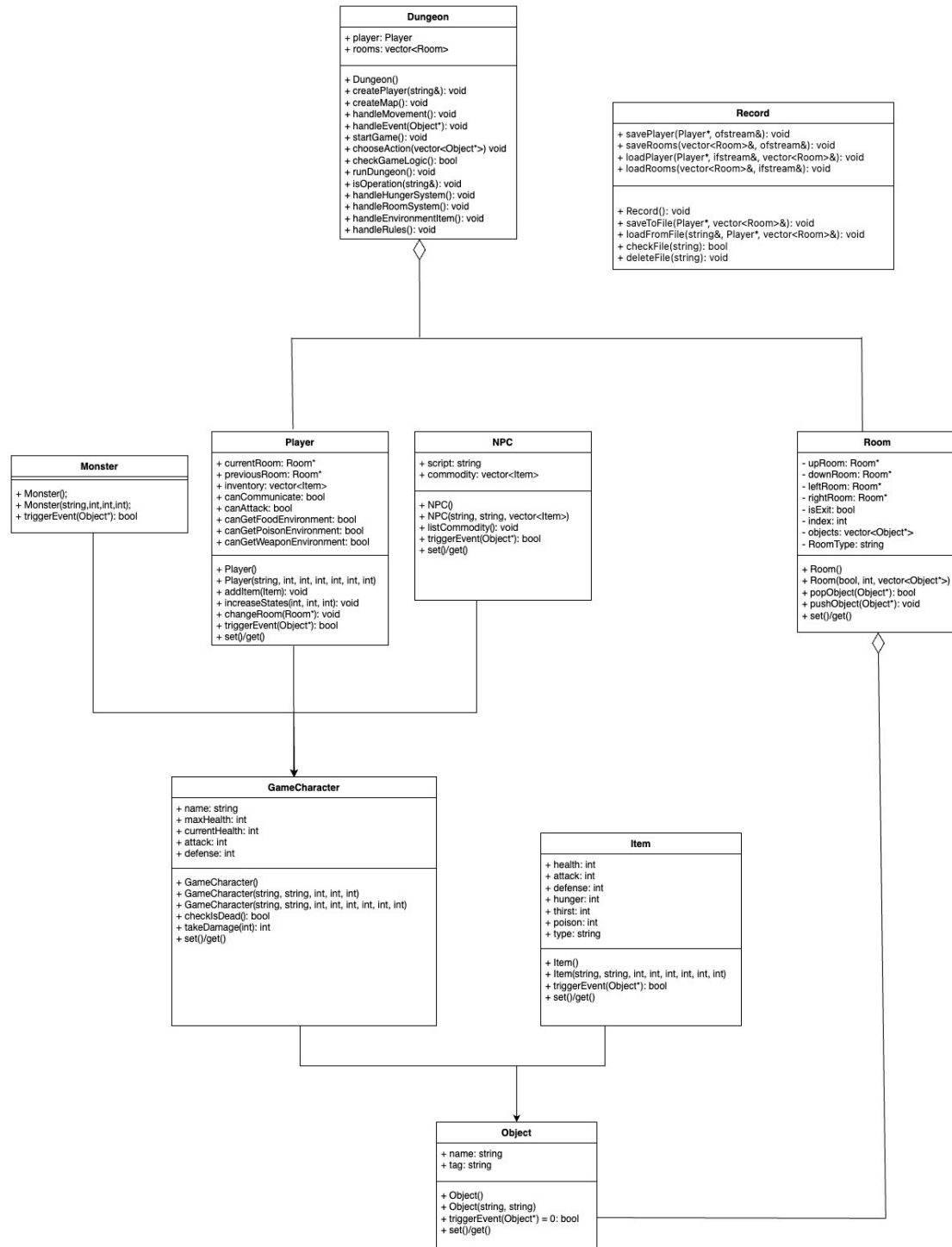
Second, Use the provided Makefile for compilation. Run the following command in your terminal:

```
make
```

Here are all the necessary files (All files are located in the same directory):

Header files	Source files	Other
Dungeon.h	main.cpp	Monster.txt
Monster.h	Dungeon.cpp	NPC.txt
Player.h	Monster.cpp	map.txt
NPC.h	Player.cpp	makefile
Record.h	NPC.cpp	
Room.h	Record.cpp	
Item.h	Room.cpp	
Object.h	Item.cpp	
GameCharacter.h	Object.cpp	
	GameCharacter.cpp	

II. UML Design



III. Basic setting and introduction

First, use the `void Dungeon::createMap()` function to read the `map.txt` file. Next, continue by reading the NPC.txt file to place NPCs in the rooms. Finally, read the monster.txt file. This completes the basic setup of the game background and room configurations.

File	Explanation
<pre>≡ map.txt 1 16 2 normal -1 3 -1 -1 3 swamp 13 -1 -1 2 4 desert -1 5 1 3 5 normal 0 -1 2 4 6 forest -1 6 3 -1 7 library 2 8 -1 -1 8 normal 4 -1 -1 7 9 swamp -1 10 6 -1 10 normal 5 11 -1 9 11 desert -1 12 8 -1 12 normal 7 14 -1 11 13 desert 8 -1 10 -1 14 swamp 9 15 -1 13 15 classroom -1 1 12 14 16 forest 10 -1 13 -1 17 normal 12 -1 -1 -1</pre>	▪ The number "16" represents that there are a total of 16 rooms. ▪ Following that, there are 16 lines, each representing a room. ▪ Taking "swamp 13 -1 -1 2" from the third line as an example, it indicates that room 1 is a swamp room, with room 13 above, has no rooms below or to the left, and room 2 on its right.
<pre>≡ NPC.txt 1 7 2 1 3 Blaze 4 The N-bomb is the best weapon. If 5 3 6 Sword Weapon 10 10 10 0 0 0 7 RedBanana Food 0 0 0 20 0 0 8 SuperWater Drink 0 0 0 0 50 0 9 3 10 Luna 11 Be careful when you step into the 12 1 13 OrangeBanana Food 0 0 0 30 0 0 14 5 15 Rex 16 Are you poisoned? Milk can help y 17 2 18 Milk Drink 0 0 0 0 30 -50 19 YellowBanana Food 0 0 0 30 0 0 20 6 21 Ivy 22 The greenbanana sounds really del 23 2 24 Spear Weapon 20 20 10 0 0 0 25 GreenBanana Food 0 0 0 10 10 10 26 0</pre> (partial content)	▪ 7 indicates that there are seven NPCs in total. ▪ Following this, for each of the seven NPCs, the process is repeated. ▪ Taking the first loop as an example, 1 represents that the NPC is located in room 1. Blaze is the name of the NPC, and "The N-bomb..." is their script. The subsequent number 3 indicates that there are three items associated with this NPC. For each item, its name, type, hp, attack, defense, hunger, thirst, and poison values are read in sequence.

After creating the map, use the `void Dungeon::createPlayer(string& theName)` function to create the player. Set the default values for the player.

(There are two kinds of character)

```
void Dungeon::createPlayer(string& theName){
    srand(time(NULL));
    bool possibility = rand() % 2;
    if(possibility == true){
        this->player = Player(theName, 100, 10, 50, 50, 50, 0);
        cout << "You are a student" << endl;
        this_thread::sleep_for(chrono::seconds(1));
        player.triggerEvent(&player);
    }
    else{
        this->player = Player(theName, 100, 50, 10, 100, 100, 0);
        cout << "You are a warrior" << endl;
        this_thread::sleep_for(chrono::seconds(1));
        player.triggerEvent(&player);
    }
    this->player.setCurrentRoom(&rooms[0]);
    this->player.setPreviousRoom(&rooms[0]);
}
```

There are six types of status:

- HP (0~100): Represent the character's current health.
- Attack (0~infinite): Represent the damage (HP) the character can make to the opponent.
- Defense (0~infinite): Represent the damage (HP) the character can reduce itself.
- Hunger (0~200): Higher means the player is not hungry, if it drops to 0, your HP will decrease.
- Thirst (0~200): Higher means the player is not thirsty, if it drops to 0, your HP will decrease.
- Poison (0~infinite): If it is not 0, it means the player is poisoned and your HP will decrease.

There are seven types of operation:

S(s): Show status <pre> rae's status: HP: 40/100 Attack: 20 Defense: 10 Hunger: 150 Thirst: 150 Poison: 0 </pre>	When the player chooses "Show status", the game will display the player's current status, including HP, attack, defense, hunger, thirst, and poison.
O(o): Open backpack <pre> You have: Sword You ate and drank: OrangeBanana apple </pre>	When choosing "Open backpack", it will display the weapons, food, drinks, and poison that the player has, ate, and drank.
M(m): Move <pre> D(d): Go down room L(l): Go left room R(r): Go right room Enter your movement: </pre>	When choosing "Move", it will display the possible moves that the player can make, and the player can select their desired move.
T(t): Talk with NPC and get items <pre> Zephyr: Hello! I'm Zephyr Zephyr: You finally found the N-Bomb! Quickly pick it up and Zephyr: Here are the items I have. Which one do you want? 0. N-bomb HP:30 Attack:70 Defense:20 1. VioletBanana Hunger:10 Thirst:10 Enter the item number you want(You can only pick one item): </pre>	When choosing "Talk with NPC and get item", it will display the NPC along with their script and the items they offer to the player. The player can then choose the corresponding number of the desired item.
A(a): Attack the monster <pre> Are you sure you are going to attack Littlemonster C(c): Confirm attack R(r): retreat Enter your decision: </pre>	When choosing "Attack the monster", the player will be prompted to either confirm the attack or retreat (returning to the previous room). The player can then make their choice accordingly.
G(g): Give up the game, and go back to study (Optional enhancement) <pre> You have to go back to study! So you give up the adventure. </pre>	When selecting "Give up the game, and go back to study", if a record exists, it will be deleted, and the game will immediately stop.
E(e): Exit the game and save record (Optional enhancement) <pre> Saving the record... Finished </pre>	If choosing "Exit the game and save record", the record will be saved before stopping the game. You can load the record back by inputting the same player's name next time.

The selection of operations depends on the objects present in the room and specific conditions.

- “Show status”, “Open backpack”, “Move”, “Exit the game and save record” appear every time the player have to pick an operation.
- “Talk with NPC and get items” appears only when there is an NPC in the room.
- “Attack the monster” appears only when there is a monster in the room.
- “Give up the game, and go back to study” is an operation that will only player enter the classroom (a special room type).

Here are some examples of some operation that you can choose in different room:

<pre> * * * * * * * * * * * * * * * * * * * * S(s): Show status O(o): Open backpack M(m): Move E(e): Exit the game and save record Enter the operation: █ </pre>	<pre> * * * * * * classroom * * * * NPC * * * * * * * * * * S(s): Show status O(o): Open backpack M(m): Move T(t): Talk with NPC and get items G(g): Give up the game, and go back to study. E(e): Exit the game and save record Enter the operation: █ </pre>
<pre> * * * * * * * * Monster * * * * * * * * * * S(s): Show status O(o): Open backpack M(m): Move A(a): Attack the monster E(e): Exit the game and save record Enter the operation: █ </pre>	<pre> * * * * * * * * NPC * * * * * * * * * * S(s): Show status O(o): Open backpack M(m): Move T(t): Talk with NPC and get items E(e): Exit the game and save record Enter the operation: █ </pre>

IV. Implementation Details

I. Movement

The player can choose the operation “M” or “m” for “Move” to move between the rooms. Handle movement using the `void Dungeon::handleMovement()` function, allowing the player to move between different rooms. This is achieved through six Room* pointers: current, previous, up, down, right, and left, which store information about the player's current room, the previous room, and the adjacent rooms in all directions. The movement function controls the player's movement within rooms where pathways exist. At the same time, it also prohibits incorrect movement commands.

The code:

```
if(operation == "m"){
    this->handleMovement();
    this->handleEnvironmentItem();
}
```

```
void Dungeon::handleMovement(){
    Room* current = player.getCurrentRoom();
    Room* previous = player.getPreviousRoom();

    Room* up = (*current).getUpRoom();
    Room* down = (*current).getDownRoom();
    Room* left = (*current).getLeftRoom();
    Room* right = (*current).getRightRoom();

    if (up != NULL) cout << "U(u): Go up room" << endl;
    if (down != NULL) cout << "D(d): Go down room" << endl;
    if (left != NULL) cout << "L(l): Go left room" << endl;
    if (right != NULL) cout << "R(r): Go right room" << endl;
    cout << "Enter your movement: ";

    string movement;
    cin >> movement;
    movement[0] = tolower(movement[0]);
    if(movement == "u" && up != NULL) player.setCurrentRoom(up);
    else if(movement == "d" && down != NULL) player.setCurrentRoom(down);
    else if(movement == "l" && left != NULL) player.setCurrentRoom(left);
    else if(movement == "r" && right != NULL) player.setCurrentRoom(right);
    else{
        cout << "You can't do this!" << endl;
    }
}
```

II. Showing Status

Showing status is divided into two different operations. The first one is to show the player's status. When selecting the operation, the player can choose "S" or "s" to display the current status of the player, including HP, attack, defense, hunger, thirst, and poison. The other operation is "open backpack." The player can choose "O" or "o". Selecting this operation will display the weapons obtained by the player, as well as the food, drink, and poison consumed.

The example of show status and open backpack:

Show status	Open backpack
<pre>rae's status: HP: 40/100 Attack: 20 Defense: 10 Hunger: 150 Thirst: 150 Poison: 0</pre>	<pre>You have: Sword You ate and drank: OrangeBanana apple</pre>

To implement showing status, the `bool Player::triggerEvent(Object* objs)`. This function is responsible for displaying the player's status and is defined as a virtual function in the Object class. Each subclass (Player, NPC, Monster) can override this function to provide different functionalities. In the Player class, the function will display the player's status when invoked.

```
//show the status of player
bool Player::triggerEvent(Object* objs){
    cout << getName() << "'s status: " << endl;
    cout << "HP: " << getCurrentHealth() << '/' << getMaxHealth() << endl;
    cout << "Attack: " << getAttack() << endl;
    cout << "Defense: " << getDefense() << endl;
    cout << "Hunger: " << getHunger() << endl;
    cout << "Thirst: " << getThirst() << endl;
    cout << "Poison: " << getPoison() << endl;
    return true;
}
```

When executing the "open backpack" operation, it iterates through the player's inventory, which is represented as a vector of **Item**. Based on the type of each item, if it is categorized as a "Weapon", it will be listed under "You have:". If it belongs to any other category, it will be listed under "You ate and drank:". If there are no items in a particular category, it will display "you have nothing" under that category.

```
if(operation == "o"){
    vector<Item> v = player.getInventory();
    int weaponCount = 0, foodCount = 0;
    if(v.size() == 0) cout << "You have nothing!" << endl;
    else{
        cout << "You have: " << endl;
        for(auto i: v){
            if(i.getType() == "Weapon"){
                cout << i.getName() << endl;
                weaponCount++;
            }
        }
        if(weaponCount == 0) cout << "nothing!!!" << endl;
        cout << "You ate and drank: " << endl;
        for(auto i: v){
            if(i.getType() != "Weapon") {
                cout << i.getName() << endl;
                foodCount++;
            }
        }
        if(foodCount == 0) cout << "nothing!!!" << endl;
    }
}
```

III. NPC

I have mentioned of how to load NPC in the room before and here are the details of the NPC and the Items it can provide.

NPC details (The script details are in NPC.txt):

Room	NPC's name	NPC's item (name, type, HP, attack, defense, hunger, thirst, poison)
1	Blaze	Sword Weapon 10 10 10 0 0 0 RedBanana Food 0 0 0 20 0 0 SuperWater Drink 0 0 0 0 50 0
3	Luna	OrangeBanana Food 0 0 0 30 0 0
5	Rex	Milk Drink 0 0 0 0 30 -50 YellowBanana Food 0 0 0 30 0 0
6	Ivy	Spear Weapon 20 20 10 0 0 0 GreenBanana Food 0 0 0 10 10 10
9	Aurora	Blade Weapon 30 30 20 0 0 0 Water Drink 0 0 0 0 20 0 BlueBanana Food 0 0 0 20 0 0
10	Ember	Axe Weapon 10 40 10 0 0 0 IndigoBanana Food 0 0 0 30 0 0 SuperMilk Drink 0 0 0 0 50 -100
13	Zephyr	N-bomb Weapon 30 70 20 0 0 0 VioletBanana Food 0 0 0 10 10 20

When executing the "Talk with NPC and get Item" operation, by typecasting the object pointer to NPC pointer, you can then use the `getScript` function to output what the NPC has to say (relevant hints or dialogues related to the game).

And then use the virtual function `triggerEvent` for NPC to manage the interaction between the player and NPC.

```
if(operation == "t"){
    NPC* NpcInRoom = dynamic_cast<NPC*>(firstobject);
    //cout << "NPC's name: " << NpcInRoom->getName() << endl;
    cout << NpcInRoom->getName() << ": Hello! I'm " << NpcInRoom->getName() << endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));
    cout << NpcInRoom->getName() << ": " << NpcInRoom->getScript() << endl;
    std::this_thread::sleep_for(std::chrono::seconds(2));
    cout << NpcInRoom->getName() << ": Here are the items I have. Which one do you want?" << endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));
    if(NpcInRoom->triggerEvent(&player)){
        player.getCurrentRoom()->popObject(NpcInRoom);
    }
}
```

IV. Pick up Items

There are two ways to pick up items. The first and most common used one is by getting it from NPC. In order to get items from NPC, the player have to choose "Talk with NPC and get Item". After talking with NPC, the NPC will list the items it has and ask you to pick item (one at a time). The second one is surprisingly getting a chance to pick up item when the player enter the specific rooms.

- Use the `triggerEvent` function in NPC class to deal with transaction:

First the function will list the items the NPC has (with serial number in front). If the item is a weapon, it will list the value HP, attack, and defense after the item's name. If the item is not weapon it will list the hunger, thirst (player won't see poison, is a secret). Then use the `addItem` function in Player class to equip the player with the item. And set the rest of the items back to NPC.

```
//deal with transaction
bool NPC::triggerEvent(Object* obj){
    Player* player = dynamic_cast<Player*>(obj);
    if(player == NULL) return false;

    vector<Item> tmp;
    vector<Item> list = commodity;
    for(int i=0; i<list.size(); i++){
        cout << i << ". " << list[i].getName();
        if(list[i].getType() == "Weapon"){
            cout << " HP:" << list[i].getHealth() << " Attack:" << list[i].getAttack() << " Defense:" << list[i].getDefense() << endl;
        }
        else{
            cout << " Hunger:" << list[i].getHunger() << " Thirst:" << list[i].getThirst() << endl;
        }
    }
    cout << "Enter the item number you want(You can only pick one item): ";
    int theItemNumber;
    cin >> theItemNumber;
    tmp.clear();
    bool picked = false;
    for(int i=0; i<list.size(); i++){
        if(i == theItemNumber){
            cout << "You picked up " << list[i].getName() << endl;
            player->addItem(list[i]);
            picked = true;
        }
        else{
            tmp.push_back(list[i]);
        }
    }
    if(picked){
        this->setCommodity(tmp);
        return(this->commodity.size() == 0);
    }
    cout << "There is no Item number " << theItemNumber << " for you to pick." << endl;
    return false;
}
```

- Pick up item surprisingly:

The code below is a part in the `HandleEnvironmentItem` function in Dungeon class, which is used to handle the item related to the room. The player can have a choice in the specific room whether he wants to pick up the item or not. The player only gets a chance when he enters the rooms. If the player picks up the Item, then the item can no longer be picked up again.

```
if(player.getCurrentRoom()->getIndex() == 2 && player.getCanGetFoodEnvironment() == true){
    cout << "-----" << endl;
    cout << "Wow! There is an apple on the floor!" << endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));
    cout << "You can pick it up if you want!" << endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));
    int choice;
    cout << "Do you want to pick it up?" << endl;
    cout << "0. Yes" << endl;
    cout << "1. No" << endl;
    cout << "Your choice:";
    cin >> choice;
    if(choice == 0){
        cout << "You get a food!" << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        cout << "Eat..." << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        cout << "The apple is very yummy." << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        cout << "Your hunger increased by 50, and your thirst increased by 30." << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        player.addItem(Item("apple", "food", 0, 0, 0, 50, 30, 0));
        player.setCanGetFoodEnvironment(false);
    }
    else if(choice == 1){
        cout << "You missed a great chance to get food!" << endl;
        this_thread::sleep_for(std::chrono::seconds(1));
    }
    else{
        cout << "invalid input(s)!" << endl;
    }
}
```

V. Fighting System

I have mentioned of how to load Monsters in the room before and here are the details of the Monsters.

Monster details:

Room	Monster's name	HP, attack, defense	Other function
4	Swampmonster	100 50 20	
8	Littlemonster	50 10 0	Poison player, feed player
15	Boss	100 100 50	Win the Boss, player win

When executing the "Attack the monster" operation, by typecasting the object pointer to Monster pointer, then use the virtual function `triggerEvent` for Monster to manage the combat between the player and Monster.

It utilizes a while loop to allow the player to choose between two actions: "confirm attack" or "retreat"(send back to previous room). If the player chooses to confirm the attack, they will initiate an attack against the monster. The function will then modify the various attributes of the monster accordingly and check if the monster is still alive. If the monster is alive, it will retaliate by attacking the player. Similarly, the function checks if the player is still alive after the monster's attack. After each round of combat, the player can choose whether to retreat or continue attacking. If the player dies, the game ends, and the game record (detailed later) is deleted. If the monster dies, different actions are taken depending on which monster it is.

```
if(this->checkIsDead()){
    //If the Boss is dead finish the game, the player win
    if(this->getName() == "Boss"){
        cout << "The Boss is dead!" << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        cout << "You win!!!" << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        Record record;
        record.deleteFile(player->getName());
        exit(0);
    }
    //other monster continue the game
    else if(this->getName() == "Littlemonster"){
        cout << "The monster(" << this->getName() << ") is dead" << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        cout << "You beat the Littlemonster." << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        cout << "You are such a hero." << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        cout << "You are going to eat the superbanana that the Littlemonster left." << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        cout << "Eat..." << endl;
        player->addItem(Item("superbanana", "food", 50, 0, 0, 50, 50, 0));
        std::this_thread::sleep_for(std::chrono::seconds(1));
        cout << "Your hunger, thirst, and HP all increased by 50." << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        return true;
    }
    else if(this->getName() == "Swampmonster"){
        cout << "The monster(" << this->getName() << ") is dead" << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        cout << "Good job!" << endl;
    }
}
```

VI. Game Logic

The victory condition in this dungeon game is solely defeating the final boss, which is the only way to win. Through various conditions set in the game (such as parameter settings, room configurations, item setups, room system, and hunger system), it's impossible to reach the final room and defeat the boss without taking any actions, except movement. As for losing the dungeon game (when the player's HP drops to zero), there are two possibilities. One is due to either not picking up appropriate items or encountering adverse conditions in various rooms (which will be detailed in the room system). Another possibility is being defeated by a monster during combat.

After each operation, the player's survival (HP not equal to zero) can be checked using the `checkGameLogic` function in the `Dungeon` class. If the HP is zero, the game ends and the record is deleted.

```
bool Dungeon::checkGameLogic(){
    if((this->player).checkIsDead()){
        cout << "-----" << endl;
        cout << "You don't have HP" << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        cout << "Game Over!" << endl;
        cout << "Game Over!!" << endl;
        cout << "Game Over!!!" << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        Record record;
        record.deleteFile(this->player.getName());
        exit(0);
        //return false;
    }
    return true;
}
```

Another point where the player's survival needs to be checked is in the `triggerEvent` function of the Monster class.

```
//deal with player's statue, and check if player is dead or not
int damageCause = player->takeDamage(this->getAttack());
cout << "The monster(" << this->getName() << ") attack you, cause " << damageCause << " damage." << endl;
std::this_thread::sleep_for(std::chrono::seconds(1));
if(player->checkIsDead()){
    cout << "You are dead, beaten by the monster(" << this->getName() << ")!" << endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));
    cout << "Game Over!!!" << endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));
    Record record;
    record.deleteFile(player->getName());
    //break;
    exit(0);
}
```

As for winning, victory is achieved automatically when the boss is defeated.

[illegible]

VII. Hunger System

There are three types of status in hunger system (hunger, thirst, poison).

`handleHungerSystem` function in Dungeon class is used to check these three statuses every time after the operation.

```
void Dungeon::handleHungerSystem(){
    cout << "-----" << endl;
    player.setHunger(player.getHunger() - 5);
    cout << "You are hungry, so your hunger decreases 5." << endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));
    player.setThirst(player.getThirst() - 5);
    cout << "You are thirsty, so your thirst decreases 5." << endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));
    if(player.getHunger() == 0){
        cout << "-----" << endl;
        cout << "Your hunger is 0!" << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        cout << "Because you are too hungry, it's very unhealthy." << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        cout << "So your HP decreased by 10." << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        player.setCurrentHealth(player.getCurrentHealth()-10);
    }
    if(player.getThirst() == 0){
        cout << "-----" << endl;
        cout << "Your thirst is 0!" << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        cout << "Because you are too thirsty, it's very unhealthy." << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        cout << "So your HP decreased by 10." << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        player.setCurrentHealth(player.getCurrentHealth()-10);
        cout << "Because you are too thirsty, you don't have power to attack." << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        cout << "So your Attack decreased by 10." << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        player.setCurrentHealth(player.getAttack()-10); //debuff
    }
    if(player.getPoisn() != 0){
        cout << "-----" << endl;
        cout << "Your poisn is NOT 0!" << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        cout << "Because you are still poisned right now." << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        cout << "It really affects your health." << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        cout << "So your HP decreased by" << player.getPoisn() << "." << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        player.setCurrentHealth(player.getCurrentHealth()-player.getPoisn()); //decrease same amount
        cout << "-----" << endl;
        cout << "As time goes by, your poisn decreases by 10." << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        player.setPoisn(player.getPoisn()-10); //duration time
        cout << "You gradually become less poisned." << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
    }
}
```

Both hunger and thirst will decrease by 5 after every operation.

Hunger:

- The default values vary depending on the player character type (warrior: 100, student: 50).
- The hunger value higher means the player is not hungry, if it drops to 0, the player HP will decrease. (Using `handleHungerSystem`)
- Three ways to get food:
 - From NPC (e.g., RedBanana, OrangeBanana...):
The player can get food from NPC (talk with NPC and get item)
 - From room (e.g., Apple)
The player can have a choice in the specific room whether he wants to pick up the item or not. (Using `HandleEnvironmentItem`)

```
if(player.getCurrentRoom()->getIndex() == 2 && player.getCanGetFoodEnvironment() == true){
    cout << "-----" << endl;
    cout << "Wow! There is an apple on the floor!" << endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));
    cout << "You can pick it up if you want!" << endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));
    int choice;
    cout << "Do you want to pick it up?" << endl;
    cout << "0. Yes" << endl;
    cout << "1. No" << endl;
    cout << "Your choice:";
    cin >> choice;
    if(choice == 0){
        cout << "You get a food!" << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        cout << "Eat..." << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        cout << "The apple is very yummy." << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        cout << "Your hunger increased by 50, and your thirst increased by 30." << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        player.addItem(Item("apple", "food", 0, 0, 0, 50, 30, 0));
        player.setCanGetFoodEnvironment(false);
    }
    else if(choice == 1){
        cout << "You missed a great chance to get food!" << endl;
        this_thread::sleep_for(std::chrono::seconds(1));
    }
    else{
        cout << "invalid input(s)!" << endl;
    }
}
```

- From Monster (e.g. SuperBanana)
The player will get food from the monster when the monster is dead. (Using `triggerEvent` in Monster class)

```
else if(this->getName() == "Littlemonster"){
    cout << "The monster(" << this->getName() << ") is dead" << endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));
    cout << "You beat the Littlemonster." << endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));
    cout << "You are such a hero." << endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));
    cout << "You are going to eat the superbanana that the Littlemonster left." << endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));
    cout << "Eat..." << endl;
    player->addItem(Item("SuperBanana", "food", 50, 0, 0, 50, 50, 0));
    std::this_thread::sleep_for(std::chrono::seconds(1));
    cout << "Your hunger, thirst, and HP all increased by 50." << endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));
    return true;
}
```

Thirst:

- The default values vary depending on the player character type (warrior: 50, student: 100).
- The hunger value higher means the player is not hungry, if it drops to 0, the player HP will decrease 10. The debuff of thirst being 0, the player attack will decrease 10. (Using `handleHungerSystem`)

```
if(player.getThirst() == 0){
    cout << "-----" << endl;
    cout << "Your thirst is 0!"<< endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));
    cout << "Because you are too thirsty, it's very unhealthy." << endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));
    cout << "So your HP decreased by 10." << endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));
    player.setCurrentHealth(player.getCurrentHealth()-10);
    cout << "Because you are too thirsty, you don't have power to attack." << endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));
    cout << "So your Attack decreased by 10." << endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));
    player.setCurrentHealth(player.getAttack()-10); //debuff
}
```

- 3 Ways to acquire water:
 - From NPC:
The player can get water from NPC (talk with NPC and get item)
 - From lake (Using `handleRoomSystem`):
The player may encounter lake in the forest, and the thirst will increase 20.

```
else{
    cout << "WOW! You find a lake in the forest." << endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));
    cout << "You aren't thirsty anymore, so your thirst increased by 20." << endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));
    player.setThirst(player.getThirst()+20);
}
```

- From oasis (Using `handleRoomSystem`):
The player may encounter oasis in the desert, and the thirst will increase 50.

```
else{ //from 0~2 oasis
    cout << "WOW! You find an oasis!" << endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));
    cout << "You can drink as much water as you want!" << endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));
    cout << "You aren't thirsty anymore, so your thirst increased by 50." << endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));
    player.setThirst(player.getThirst()+50);
}
```

Poison:

- The default value of poison is 0.
- The poison value higher means the player is poisoned, if it is not 0, the player HP will decrease the same amount of poison.
- 3 ways to get poisons:

- From NPC (e.g. GreenBanana, VioletBanana)

The Player may accidentally get the poison from NPC, because these poisons are disguised as food. The player will think it is food and might pick it up.

(talk with NPC and get item)

- From room (e.g. PoisonBanana)

The player will pick up the food on the floor and eat it but find out it was poison afterwards. (Using `HandleEnvironmentItem`)

```
if(player.getCurrentRoom()->getIndex() == 4 && player.getCanGetPoisonEnvironment() == true){
    player.addItem(Item("PoisonBanana", "poison", 0, 0, 0, 10, 0, 20));
    cout << "-----" << endl;
    cout << "You accidentally picked up a PoisonBanana on the floor." << endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));
    cout << "And you ate it." << endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));
    cout << "You were poisoned by the PoisonBanana." << endl;
    std::this_thread::sleep_for(std::chrono::seconds(1)); |
    cout << "Your poison increased by 20." << endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));
    cout << "At least you can fill your stomach." << endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));
    cout << "Your hunger also increased by 10." << endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));
    player.setCanGetFoodEnvironment(false);
}
```

- From Monster (e.g. devilpoison)

The player will be forced to drink the poison if he chooses to attack the monster. (Using `triggerEvent` in Monster class)

```
//get poison from the monster
if(this->getName() == "Littlemonster" && this->getCurrentHealth() == 50 ){
    player->addItem(Item("devilpoison", "poison", 0, 0, 0, 0, 0, 10));
    cout << "The Littlemonster forced you to drink the Devil Poison." << endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));
    cout << "Your poison increased by 10." << endl;
    std::this_thread::sleep_for(std::chrono::seconds(2));
}
```

- Detoxify

- Can acquire milk from NPC to detoxify. In room5 (milk) and room10 (supermilk), supermilk is a better antidote.
- May find rare plants in the swamp, can also detoxify. (Using `handleRoomSystem`)

VIII. Room System

The `handleRoomSystem` function in the Dungeon class is responsible for most of the operations related to the room system. There are five types of rooms: desert, forest, swamp, library, and classroom.

Most of the event happen randomly:

```
srand(time(NULL));  
int possibility = rand() % 10; //from 0~9
```

Desert:

```
if(player.getCurrentRoom()->getRoomType() == "desert"){  
    cout << "-----" << endl;  
    cout << "You are in the desert right now." << endl;  
    std::this_thread::sleep_for(std::chrono::seconds(1));  
    if(possibility>=6){ //from6~9 sandstorm  
        cout << "OOPS! You're in a sandstorm." << endl;  
        std::this_thread::sleep_for(std::chrono::seconds(1));  
        cout << "It's really dangerous!" << endl;  
        std::this_thread::sleep_for(std::chrono::seconds(1));  
        cout << "And you can't find any water" << endl;  
        std::this_thread::sleep_for(std::chrono::seconds(1));  
        cout << "You are very thirsty, so you're thirst reduced by 20." << endl;  
        std::this_thread::sleep_for(std::chrono::seconds(1));  
        player.setThirst(player.getThirst()-20);  
    }  
    else if(possibility<=5 && possibility>=2){ //from2~5 just desert  
        cout << "The desert is very hot." << endl;  
        std::this_thread::sleep_for(std::chrono::seconds(1));  
        cout << "You can't find any water." << endl;  
        std::this_thread::sleep_for(std::chrono::seconds(1));  
        cout << "You are thirsty, so your thirst reduced by 10." << endl;  
        std::this_thread::sleep_for(std::chrono::seconds(1));  
        player.setThirst(player.getThirst()-10);  
    }  
    else{ //from0~2 oasis  
        cout << "WOW! You find an oasis!" << endl;  
        std::this_thread::sleep_for(std::chrono::seconds(1));  
        cout << "You can drink as much water as you want!" << endl;  
        std::this_thread::sleep_for(std::chrono::seconds(1));  
        cout << "You aren't thirsty anymore, so your thirst increased by 50." << endl;  
        std::this_thread::sleep_for(std::chrono::seconds(1));  
        player.setThirst(player.getThirst()+50);  
    }  
}
```

- Thirst decrease: When $5 \geq \text{possibility} \geq 2$, thirst reduced by 10 (compare to normal: 5).
- Sandstorms: When $9 \geq \text{possibility} \geq 6$, thirst reduced by 20 (compare to normal: 5 and just desert: 10).
- Oasis: When $2 \geq \text{possibility} \geq 0$, the thirst not only won't reduce, it will increase 50.

Forest:

```
if(player.getCurrentRoom()->getRoomType() == "forest"){
    cout << "-----" << endl;
    cout << "You are in the forest right now." << endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));
    cout << "OOPS! You can't find any food in the forest." << endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));
    cout << "You are very hungry, so your hunger decreases 20." << endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));
    player.setHunger(player.getHunger()-20);
    std::this_thread::sleep_for(std::chrono::seconds(1));
    if(possibility >= 0 && possibility <= 4){
        cout << "The forest is very dangerous." << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        cout << "There are many formidable predators." << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        if(possibility >= 0 && possibility <= 2){
            cout << "A tiger is chasing you!!!" << endl;
            std::this_thread::sleep_for(std::chrono::seconds(1));
            cout << "Keep running!!!" << endl;
            std::this_thread::sleep_for(std::chrono::seconds(1));
            cout << "Run..." << endl;
            std::this_thread::sleep_for(std::chrono::seconds(1));
            cout << "Run..." << endl;
            std::this_thread::sleep_for(std::chrono::seconds(1));
            cout << "Run..." << endl;
            std::this_thread::sleep_for(std::chrono::seconds(1));
            player.setCurrentHealth(player.getCurrentHealth() - 10);
            player.setHunger(player.getHunger() - 10);
            player.setThirst(player.getThirst() - 10);
            cout << "Your HP, hunger, and thirst are each reduced by 10 because you ran vigorously." << endl;
            std::this_thread::sleep_for(std::chrono::seconds(1));
            cout << "Congratulations, you've escaped the tiger." << endl;
            std::this_thread::sleep_for(std::chrono::seconds(1));
            cout << "Keep the adventuring going!" << endl;
            std::this_thread::sleep_for(std::chrono::seconds(1));
        }
        else if(possibility >= 2 && possibility <= 4){
            cout << "A bear is chasing you!!!" << endl;
            std::this_thread::sleep_for(std::chrono::seconds(1));
            cout << "Keep running!!!" << endl;
            std::this_thread::sleep_for(std::chrono::seconds(1));
            cout << "Run..." << endl;
            std::this_thread::sleep_for(std::chrono::seconds(1));
            cout << "Run..." << endl;
            std::this_thread::sleep_for(std::chrono::seconds(1));
            cout << "Run..." << endl;
            std::this_thread::sleep_for(std::chrono::seconds(1));
            cout << "Run..." << endl;
            std::this_thread::sleep_for(std::chrono::seconds(1));
            player.setCurrentHealth(player.getCurrentHealth() - 10);
            player.setHunger(player.getHunger() - 10);
            player.setThirst(player.getThirst() - 10);
            cout << "Your HP, hunger, and thirst are each reduced by 10 because you ran vigorously." << endl;
            std::this_thread::sleep_for(std::chrono::seconds(1));
            cout << "Congratulations, you've escaped the bear." << endl;
            std::this_thread::sleep_for(std::chrono::seconds(1));
            cout << "Keep the adventuring going!" << endl;
            std::this_thread::sleep_for(std::chrono::seconds(1));
        }
    }
    else{
        cout << "WOW! You find a lake in the forest." << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        cout << "You aren't thirsty anymore, so your thirst increased by 20." << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        player.setThirst(player.getThirst()+20);
    }
}
```

- Hunger consumption: No matter what happen when a player is in the forest, it's hunger will decrease by 20. (compare to normal: 5).
- Wildlife: When $4 \geq \text{possibility} \geq 0$, the player will encounter tiger or bear, and the player have to run away. So hunger, thirst, HP all decrease by 10.
- Lake: When $9 \geq \text{possibility} \geq 5$, the thirst not only won't reduce, it will increase 20.

Swamp:

```
if(player.getCurrentRoom()->getRoomType() == "swamp"){
    cout << "-----" << endl;
    cout << "You are in the swamp right now." << endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));
    cout << "Swamp is actually full of dangers!" << endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));
    cout << "OOPS! You accidentally got poisoned." << endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));
    cout << "Your poison has increased by 10" << endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));
    player.setPoison(player.getPoison() + 10);
    if(possibility >= 0 && possibility <= 4){
        cout << "You're really lucky!" << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        cout << "You've actually found rare plants that can cure all kinds of illnesses." << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        cout << "Your poison has been reset to zero." << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        cout << "And your hunger and thirst have both increased by 50" << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        cout << "Your HP is also back to 100." << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        player.setPoison(0);
        player.setHunger(player.getHunger() + 50);
        player.setThirst(player.getThirst() + 50);
        player.setCurrentHealth(100);
    }
    else{
        cout << "You have very bad luck!" << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        cout << "You've been bitten by poisonous insects." << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        cout << "After being bitten by the insects, you're both hungry and thirsty, and you're lacking energy." << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        cout << "Your hunger and thirst have both decreased by 10" << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        cout << "And your HP has also decreased by 10." << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        player.setHunger(player.getHunger() - 10);
        player.setThirst(player.getThirst() - 10);
        player.setCurrentHealth(player.getCurrentHealth() - 10);
    }
}
```

- Poison increase: No matter what happen when a player is in the swamp, it's poison will increase by 10.
- Discovery of Rare Plants: When $4 \leq \text{possibility} \leq 0$, the player will find rare plants to cure illnesses. So, poison reset to zero, hunger, thirst increased 50, and hp go back to 100.
- Insects: When $9 \leq \text{possibility} \leq 5$, the player will be bitten by the insects. So, hunger, thirst, HP decreased by 10.

Library (Optional enhancement):

```
if(player.getCurrentRoom()->getRoomType() == "library"){
    cout << "-----" << endl;
    cout << "You are in the library right now." << endl;
    cout << "The library has a very good air conditioner." << endl;
    this_thread::sleep_for(chrono::seconds(1));
    cout << "You can take a nap." << endl;
    this_thread::sleep_for(chrono::seconds(1));
    cout << "Nap..." << endl;
    this_thread::sleep_for(chrono::seconds(1));
    cout << "Nap..." << endl;
    this_thread::sleep_for(chrono::seconds(1));
    cout << "Nap..." << endl;
    this_thread::sleep_for(chrono::seconds(1));
    cout << "You are full of energy again!" << endl;
    this_thread::sleep_for(chrono::seconds(1));
    cout << "Your HP is restored to 100." << endl;
    this_thread::sleep_for(chrono::seconds(1));
    cout << "Your hunger increased by 50." << endl;
    this_thread::sleep_for(chrono::seconds(1));
    cout << "Your thirst increased by 50." << endl;
    this_thread::sleep_for(chrono::seconds(1));
    player.setCurrentHealth(100);
    player.setHunger(player.getHunger() + 50);
    player.setThirst(player.getThirst() + 50);
}
```

- Great place to rest a bit: When being in the library, the player's HP will be restored to 100, hunger and thirst will increase by 50.

Classroom (Optional enhancement):

```
if(player.getCurrentRoom()->getIndex() == 13 && player.getCanGetWeaponEnvironment() == true){
    cout << "-----" << endl;
    cout << "You are in the classroom right now." << endl;
    this_thread::sleep_for(chrono::seconds(1));
    cout << "There is a power weapon called book!" << endl;
    this_thread::sleep_for(chrono::seconds(1));
    cout << "You can pick it up if you want!" << endl;
    this_thread::sleep_for(chrono::seconds(1));
    int choice;
    cout << "Do you want to pick it up?" << endl;
    cout << "0. Yes" << endl;
    cout << "1. No" << endl;
    cout << "Your choice:";
    cin >> choice;
    if(choice == 0){
        cout << "You get a powerful weapon!" << endl;
        player.addItem(Item("book", "Weapon", 0, 30, 30, 0, 0, 0));
        player.setCanGetWeaponEnvironment(false);
    }
    else if(choice == 1){
        cout << "You miss a great chance to get weapon!" << endl;
        this_thread::sleep_for(chrono::seconds(1));
    }
    else{
        cout << "invalid input(s)!" << endl;
    }
}
```

- Powerful room: When being in the classroom, the player gets to choose if he want to pick up the weapon called book or not if he hasn't pick it up yet. (Using 'handleEnvironmentItem')

```
if(operation == "g"){
    Record record;
    cout << "You have to go back to study!" << endl;
    this_thread::sleep_for(chrono::seconds(1));
    cout << "So you give up the adventure." << endl;
    this_thread::sleep_for(chrono::seconds(1));
    record.deleteFile(player.getName());
    exit(0);
}
```

- Give up: Compare to other room, only when player is in the classroom, he has the opportunity to choose 'Give up the game, and go back to study'. If the player chooses this operation than the game will stop. If the record exists than it will be deleted.

IX. Record (Optional enhancement)

The primary concept behind maintaining the record is to store all the necessary information required to continue the game in a file. I've divided it into two files.

"playername_map.txt" is used to store all the information about the map, while

"playername_player.txt" is used to store information about the player.

(All the function mentioned in this part is in Record class)

Use 'savePlayer' to output all the player's information in "playername_player.txt", and use 'saveRooms' to output all the map's information (including rooms, NPCs, Monsters...) in "playername_map.txt". Finally, use 'saveToFile' to execute both functions ('savePlayer', 'saveRooms').

Use 'loadPlayer' to load all the player's information back in "playername_player.txt", and use 'loadRooms' to load all the map's information (including rooms, NPCs, Monsters...) back in "playername_map.txt". Finally, use 'loadFromFile' to execute both functions ('loadPlayer', 'loadRooms').

The Record class utilizes the concept of ifstream and ofstream to establish a correspondence between output and input, thereby achieving its functionality.

```
class Record
{
private:
    void savePlayer(Player*, ofstream&);
    void saveRooms(vector<Room>&, ofstream&);
    void loadPlayer(Player*, ifstream&, vector<Room>&);
    void loadRooms(vector<Room>&, ifstream&);

public:
    Record();
    void saveToFile(Player*, vector<Room>&);
    void loadFromFile(string&, Player*, vector<Room>&);

    bool checkFile(string);
    void deleteFile(string);
};
```

The only way to save file is by select 'Exit the game and save record', when the player chooses the operation.

```
if(operation == "e"){
    Record record;
    record.saveToFile(&player, rooms);
    break;
}
```

When the game start, will check if the file with the name player input exists or not (Using `checkFile`).

- If the player already exists, then will ask player do he want to continue. If the player wants to continue than the file will be load back. If not, the player has to enter a new name.
- If the player doesn't exist, then the game will just start.

```
if(record.checkFile(player_name)){
    string operation;
    cout << "Found previous records." << endl;
    cout << "Do you want to continue?(Enter y for yes, n for no):";
    cin >> operation;
    operation[0] = tolower(operation[0]);
    if(operation == "y"){
        record.loadFromFile(player_name, &player, rooms);
        std::this_thread::sleep_for(std::chrono::seconds(1));
        cout << "Hello! " << player_name << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        cout << "Welcome back!" << endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));

        canStart = true;
    }
    else if(operation == "n"){
        cout << "You have to change a name!" << endl;
    }
    else{
        cout << "Invalid input(s)" << endl;
    }
}
else{
    cout << "Hello! " << player_name << endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));
    this->createMap();
    this->createPlayer(player_name);
    canStart = true;
}
```

The record will be deleted under three circumstances:

- The player wins the game.
- The player loses.
- The player gives up.

v. Optional Enhancement

1. Record system

(Refer to the implementation details in the Record section for more information.)

2. Player character

Player may be two types of character randomly. Which is student or warrior, they have different status value at the beginning.

```
void Dungeon::createPlayer(string& theName){
    srand(time(NULL));
    bool possibility = rand() % 2;
    if(possibility == true){
        this->player = Player(theName, 100, 10, 50, 50, 50, 0);
        cout << "You are a student" << endl;
        this_thread::sleep_for(chrono::seconds(1));
        player.triggerEvent(&player);
    }
    else{
        this->player = Player(theName, 100, 50, 10, 100, 100, 0);
        cout << "You are a warrior" << endl;
        this_thread::sleep_for(chrono::seconds(1));
        player.triggerEvent(&player);
    }
    this->player.setCurrentRoom(&rooms[0]);
    this->player.setPreviousRoom(&rooms[0]);
}
```

3. Give up option

The player can choose to give up in the specific room (classroom) when they enter. The game will stop immediately and delete the record.

(Refer to the implementation details in the room system section for more information.)

4. Additional rooms

Library, and classroom are additional rooms.

(Refer to the implementation details in the room system section for more information.)

VI. Results

The start:

```
rae@RaedeMacBook-Air Dungeon % ./a.out
Welcome to the dungeon!
Here are the rules of the dungeon.
Before you start the adventure, you have to remember these rules!
As a player, you have two types of states: one is related to your combat ability, and the other is related to your health status.
HP represents your health points. When your HP reaches 0, the game ends.
In battles with monsters, your HP may decrease when your hunger or thirst reaches 0.
Attack represents your combat ability. You can obtain weapons from NPCs to increase your attack.
Defense represents your defensive ability. You can obtain weapons from NPCs to increase your defense.
Hunger indicates whether you are hungry. When hunger reaches 0, your HP will decrease.
You can increase your hunger by obtaining food from NPCs, defeating monsters to obtain food, or finding food in certain rooms. Hunger decreases over time.
Thirst indicates whether you are thirsty. When thirst reaches 0, your HP will decrease.
You can increase your thirst by obtaining water from NPCs or certain food items. Thirst decreases over time.
Poison indicates whether you are poisoned. When poison is not 0, you are poisoned, with higher values indicating deeper poisoning.
You may accidentally consume poisonous food from NPCs or encounter poison in certain rooms. You can cure poison by obtaining milk. Poison decreases over time after being poisoned.
In the dungeon, there are different types of rooms: desert, forest, swamp, and regular rooms.
Remember, each room may hold different surprises!
Finally, if you defeat the boss, you will achieve victory!
Good luck!

DUNGEON

Enter your name: j
```

Win:

```

Boss: I'm Boss
Boss: You really want to fight
Boss: Come on!
Your status:
HP: 70/100
Attack: 160
Defense: 70
Boss status:
HP: 100/100
Attack: 100
Defense: 50
Are you sure you are going to attack Boss
C(c): Confirm attack
R(r): retreat
Enter your decision:c
You attack the monster(Boss), cause 110 damage.
The Boss is dead!
You win!!!

      dP      dP      dP      oo      dP
      88      88      88      88      88
88      .d888b. 88d888b. .d888b. 88d888b. .d888b. d8888P dP dP 88 .d888b. d8888P dP .d888b. 88d888b. .d888b. 88
88 88' 88 88' 88 88' 88 88' 88 88 88 88 88 88' 88 88 88 88' 88 88 88 88' 88 88 88' 88 Y8ooooo. dP
Y8 .88 88 .88 88 88 88 .88 88 88 .88 88 88 .88 88 88 .88 88 88 88 88 88 88 88 88 88 88 88 88 88
Y88888P' 88888P' dP dP 88888P88 dP 88888P8 dP 88888P' dP 88888P8 dP dP 88888P' dP dP 88888P' oo
      .88
      d8888P

rae@RaedeMacBook-Air Dungeon %
```

Lose:

```
Your thirst is 0!
Because you are too thirsty, it's very unhealthy.
So your HP decreased by 10.
Because you are too thirsty, you don't have power to attack.
So your Attack decreased by 10.

You don't have HP
Game Over!!
Game Over!!!
Game Over!!!
rae@RaedeMacBook-Air Dungeon %
```

Operation:

```
*****
*   classroom   *
*               *
*       NPC      *
*               *
*               *
*****
S(s): Show status
O(o): Open backpack
M(m): Move
T(t): Talk with NPC and get items
G(g): Give up the game, and go back to study.
E(e): Exit the game and save record
Enter the operation: █
```

Show status:

```
rae's status:
HP: 40/100
Attack: 20
Defense: 10
Hunger: 150
Thirst: 150
Poison: 0
```

Open backpack:

```
You have:
Sword
You ate and drank:
OrangeBanana
apple
```

Movement:

```
D(d): Go down room
L(l): Go left room
R(r): Go right room
Enter your movement: █
```

Talk to NPC and get item:

```
Zephyr: Hello! I'm Zephyr  
Zephyr: You finally found the N-Bomb! Quickly pick it up and  
Zephyr: Here are the items I have. Which one do you want?  
0. N-bomb HP:30 Attack:70 Defense:20  
1. VioletBanana Hunger:10 Thirst:10  
Enter the item number you want(You can only pick one item):
```

Attack the Monster:

```
Are you sure you are going to attack Littlemonster  
C(c): Confirm attack  
R(r): retreat  
Enter your decision:
```

Give up:

```
You have to go back to study!  
So you give up the adventure.
```

Exit and save record:

```
Saving the record...  
Finished
```

VII. Discussion

After completing the dungeon, I believe that my dungeon is very comprehensive, meeting all the requirements including all the basic functions. Additionally, I have added some extra features, such as the record system, random selection of game characters, and additional rooms for room system.

However, there are areas where I can improve:

- I can enhance the randomness by introducing more variability in the monsters and NPCs found in the rooms, rather than having fixed configurations. To achieve this, instead of assigning fixed rooms to monsters and NPCs when loading them, I can store them in a vector and randomly allocate them to different rooms. This approach would add more excitement and challenge to the game.
- I can implement a more complex combat system beyond just player and monster interactions. To do so, I can introduce various attributes for different characters, such as water countering fire or fire countering grass, etc. Additionally, I can implement a feature where monsters become progressively stronger over time, making them more challenging to defeat.

For future improvement:

- There is an opportunity to transform the game interface from displaying only text and ASCII art in the terminal to a graphical user interface (GUI). This could be achieved by leveraging technologies such as Qt or GTK+. I can learn various skills to enable me to develop a GUI game in the future.

VIII. Conclusion

This is the first project I've completed independently since entering college, and it has been a tremendous learning experience for me.

Firstly, it has provided me with a comprehensive understanding of Object-Oriented Programming (OOP) concepts, which I applied to create the Dungeon project. Being able to apply the concepts I learned before the midterm exam and put them into practice was a valuable opportunity for me. Some concepts that were difficult to grasp solely through problem-solving and classroom lectures became clearer to me through project development. This experience has deepened my understanding of the overall concepts and enhanced my problem-solving skills.

Through this endeavor, I gained practical knowledge of software development processes and programming techniques. From setting up the environment to coding each step, I completed everything independently. Throughout the process, besides setting up the environment and coding, I also gained a deeper understanding of Git, tracking the progress of my project and ensuring the modifications were correctly implemented.

Overall, this project not only enhanced my understanding of OOP concepts but also provided me with deeper insights into various other aspects of programming.